

Chapter 2: Scale Machine Learning Data

Many machine learning algorithms expect data to be scaled consistently. There are two popular methods that you should consider when scaling your data for machine learning. In this tutorial, you will discover how you can rescale your data for machine learning. After reading this tutorial you will know:

- How to normalize your data from scratch.
- How to standardize your data from scratch.
- When to normalize as opposed to standardize data.

Let's get started.

2.1 Description

Many machine learning algorithms expect the scale of the input and even the output data to be equivalent. It can help in methods that weight inputs in order to make a prediction, such as in linear regression and logistic regression. It is practically required in methods that combine weighted inputs in complex ways such as in artificial neural networks and deep learning.

2.1.1 Pima Indians Diabetes Dataset

In this tutorial we will use the Pima Indians Diabetes Dataset. This dataset involves the prediction of the onset of diabetes within 5 years. The baseline performance on the problem is approximately 65%. You can learn more about it in Appendix A, Section A.4. Download the dataset and save it into your current working directory with the filename `pima-indians-diabetes.csv`.

2.2 Tutorial

This tutorial is divided into 3 parts:

1. Normalize Data.
2. Standardize Data.
3. When to Normalize and Standardize.

These steps will provide the foundations you need to handle scaling your own data.

2.2.1 Normalize Data

Normalization can refer to different techniques depending on context. Here, we use normalization to refer to rescaling an input variable to the range between 0 and 1. Normalization requires that you know the minimum and maximum values for each attribute. This can be estimated from training data or specified directly if you have deep knowledge of the problem domain. You can easily estimate the minimum and maximum values for each attribute in a dataset by enumerating through the values. The snippet of code below defines the `dataset minmax()` function that calculates the min and max value for each attribute in a dataset, then returns an array of these minimum and maximum values.

```
In [17]: use strict;
use warnings;
use Data::Dump qw(dump);
use List::Util qw(zip min max sum);
use sml; # Statistical Machine Learning Library
```

```
In [18]: # Function To Calculate the Min and Max Values For a Dataset.
# Find the min and max values for each column
sub dataset_minmax{
    my ($self, $dataset) = @_;
    my @minmax;
    for my $i (0 .. $#{$dataset->[0]}){ # Be careful not to include the Y labels
        my $col_values = [map {$_->[$i]} @$dataset];
        my $value_min = min(@$col_values);
        my $value_max = max(@$col_values);
        push @minmax, [$value_min, $value_max];
    }
    return \@minmax;
}

sml->add_to_class('dataset_minmax', \&{'dataset_minmax'});
```

```
Out[18]: *sml::dataset_minmax
```

Subroutine dataset_minmax redefined at reply input line 3.

With this contrived dataset, we can test our function for calculating the min and max for each column.

```
In [19]: # Contrive small dataset
my $dataset = [[50, 30], [20, 90]];
printf "Dataset: %s\n", dump $dataset;
# Calculate min and max for each column
my $minmax = sml->dataset_minmax($dataset);
printf "Minimax: %s\n", dump $minmax;
# Output of Example Calculating the Min and Max Values.
# [[50, 30], [20, 90]]
# [[20, 50], [30, 90]]
```

```
Dataset: [[50, 30], [20, 90]]
```

```
Minimax: [[20, 50], [30, 90]]
```

```
Out[19]: 1
```

Once we have estimates of the maximum and minimum allowed values for each column, we can now normalize the raw data to the range 0 and 1. The calculation to normalize a single value for a column is:

$$\text{scaled value} = (\text{value} - \text{min}) / (\text{max} - \text{min})$$

(2.1)

Below is an implementation of this in a function called normalize_dataset() that normalizes values in each column of a provided dataset.

```
In [20]: # Function To Normalize a Dataset.
# Rescale dataset columns to the range 0-1
# Be careful not to include the y labels
sub normalize_dataset{
    my ($self, $X, $minmax) = @_;
    for my $row (@$X){
        for my $pos (0 .. $#{$row}){
            $row->[$pos] = ($row->[$pos] - $minmax->[$pos][0]) / ($minmax->[$pos][1] - $minmax->[$pos][0]);
        }
    }
}
```

```

    }
}

sml->add_to_class('normalize_dataset', \&{'normalize_dataset'});

```

Out[20]: *sml::normalize_dataset

Subroutine normalize_dataset redefined at reply input line 4.

We can tie this function together with the dataset minmax() function and normalize the contrived dataset.

```

In [21]: # Contrive small dataset
$dataset = [[50, 30], [20, 90]];
printf "Dataset: %s\n", dump $dataset;
# Calculate min and max for each column
$minmax = sml->dataset_minmax($dataset);
printf "Minimax: %s\n", dump $minmax;
# Normalize columns
sml->normalize_dataset($dataset, $minmax);
printf "Normalized: %s\n", dump $dataset;

# Example Output of Normalizing the Contrived Dataset.
# [[50, 30], [20, 90]]
# [[20, 50], [30, 90]]
# [[1, 0], [0, 1]]

```

```

Dataset: [[50, 30], [20, 90]]
Minimax: [[20, 50], [30, 90]]
Normalized: [[1, 0], [0, 1]]

```

Out[21]: 1

We can combine this code with code for loading a CSV dataset and load and normalize the Pima Indians Diabetes dataset. The example first loads the dataset and converts the values for each column from string to floating point values. The minimum and maximum values for each column are estimated from the dataset, and finally, the values in the dataset are normalized.

```

In [22]: # Load pima-indians-diabetes dataset
my $filename = '../data/pima-indians-diabetes.csv';
$dataset = sml->load_csv($filename);

my $X = [map { [ @$_[0] .. ${$dataset->[0]} - 1] } @$dataset];
my $y = [map { @$_[1] } @$dataset];

printf "Loaded data file %s with %d rows and %d columns.\n\n",
    $filename, scalar(@$dataset), scalar(@{$dataset->[0]});
printf "X: %s\n\n",
    dump map {[map {sprintf "%0.0f", $_} @$_]} @$X[0 .. 4];

# Calculate min and max for each column
$minmax = sml->dataset_minmax($X);
sml->normalize_dataset($X, $minmax);
printf "Normalized: %s\n",
    dump map {[map {sprintf "%0.2f", $_} @$_]} @$X[0 .. 4];

# Example Output of Normalizing the Diabetes Dataset.
# Loaded data file pima-indians-diabetes.csv with 768 rows and 9 columns
# Dataset[0]: [6.0, 148.0, 72.0, 35.0, 0.0, 33.6, 0.627, 50.0, 1.0]
# Normalized: Normalized: (0.35, 0.74, 0.59, 0.35, "0.00", "0.50", 0.22, 0.48, "1.00")

```

Loaded data file ../data/pima-indians-diabetes.csv with 768 rows and 9 columns.

```
X: (
  [6, 148, 72, 35, 0, 34, 1, 50],
  [1, 85, 66, 29, 0, 27, 0, 31],
  [8, 183, 64, 0, 0, 23, 1, 32],
  [1, 89, 66, 23, 94, 28, 0, 21],
  [0, 137, 40, 35, 168, 43, 2, 33],
)

Normalized: (
  [0.35, 0.74, 0.59, 0.35, "0.00", "0.50", 0.23, 0.48],
  [0.06, 0.43, 0.54, 0.29, "0.00", "0.40", 0.12, 0.17],
  [0.47, 0.92, 0.52, "0.00", "0.00", 0.35, 0.25, 0.18],
  [0.06, 0.45, 0.54, 0.23, 0.11, 0.42, 0.04, "0.00"],
  ["0.00", 0.69, 0.33, 0.35, "0.20", 0.64, 0.94, "0.20"],
)
```

Out[22]: 1

2.2.2 Standardize Data

Standardization is a rescaling technique that refers to centering the distribution of the data on the value 0 and the standard deviation to the value 1. Together, the mean and the standard deviation can be used to summarize a normal distribution, also called the Gaussian distribution or bell curve. It requires that the mean and standard deviation of the values for each column be known prior to scaling. As with normalizing above, we can estimate these values from training data, or use domain knowledge to specify their values. Let's start with creating functions to estimate the mean and standard deviation statistics for each column from a dataset. The mean describes the middle or central tendency for a collection of numbers. The mean for a column is calculated as the sum of all values for a column divided by the total number of values.

$$\sum_{i=1}^n values_i / count(values)$$

(2.2)

The function below named `column_means()` calculates the mean values for each column in the dataset.

```
In [23]: # Function To Calculate Means For Each Column in a Dataset.
# Calculate column means
sub column_means{
  my ($self, $dataset) = @_;
  my $means = [0, map {$_} 0 .. ${$dataset->[0]} -1];
  for my $i (0 .. ${$dataset->[0]}){
    my $col_values = [map {$_->[$i]} @$dataset];
    $means->[$i] = sum(@$col_values) / scalar(@$dataset);
  }
  return $means;
}

sml->add_to_class('column_means', \&{'column_means'});
```

Out[23]: *sml::column_means

Subroutine `column_means` redefined at reply input line 3.

The standard deviation describes the average spread of values from the mean. It can be calculated as the square root of the sum of the squared difference between each value and the mean and dividing by

the number of values minus 1.

$$\text{standard deviation} = \sqrt{\sum_{i=1}^n (\text{values}_i - \text{mean})^2 / \text{count}(\text{values}) - 1}$$

(2.3)

The function below named `column stdevs()` calculates the standard deviation of values for each column in the dataset and assumes the means have already been calculated.

```
In [24]: # Function To Calculate Standard Deviations For Each Column in a Dataset.
# Calculate column standard deviations
sub column_stdevs{
  my ($self, $dataset, $means) = @_;
  my $stdevs = [0, map {$_} 0 .. ${#$dataset->[0]} -1];
  for my $i (0 .. ${#$dataset->[0]}){
    my $variance = [map {($_->[$i] - $means->[$i]) ** 2} @$dataset];
    $stdevs->[$i] = sum(@$variance);
  }
  $stdevs = [map {sqrt($_ / (scalar(@$dataset) -1))} @$stdevs];
  return $stdevs;
}

sml->add_to_class('column_stdevs', \&{'column_stdevs'});
```

```
Out[24]: *sml::column_stdevs
```

Subroutine `column_stdevs` redefined at reply input line 3.

Using the contrived dataset, we can estimate the summary statistics.

```
In [25]: # Standardize dataset
$dataset = [[50, 30], [20, 90], [30, 50]];
printf "%s\n", dump $dataset;
# Estimate mean and standard deviation
my $means = sml->column_means($dataset);
my $stdevs = sml->column_stdevs($dataset, $means);
printf "Means: %s\n", dump map {sprintf "%0.2f", $_} @$means;
printf "Stdevs: %s\n", dump map {sprintf "%0.2f", $_} @$stdevs;

# Example Output From Calculating Statistics from the Contrived Dataset.
# [[50, 30], [20, 90], [30, 50]]
# Means: (33.33, 56.67)
# Stdevs: (15.28, 30.55)
```

```
[[50, 30], [20, 90], [30, 50]]
Means: (33.33, 56.67)
Stdevs: (15.28, 30.55)
```

```
Out[25]: 1
```

Once the summary statistics are calculated, we can easily standardize the values in each column. The calculation to standardize a given value is as follows:

$$\text{standardized_value}_i = (\text{value}_i - \text{mean}) / \text{stdev}$$

(2.4)

Below is a function named `standardize dataset()` that implements this equation

```
In [26]: # Function To Standardize a Dataset.
# Standardize dataset
```

```

sub standardize_dataset{
  my ($self, $X, $means, $stdevs) = @_;
  for my $row (@$X){
    for my $i (0 .. $#$row){
      $row->[$i] = ($row->[$i] - $means->[$i]) / $stdevs->[$i];
    }
  }
}

sml->add_to_class('standardize_dataset', \&'standardize_dataset');

```

Out[26]: *sml::standardize_dataset

Subroutine standardize_dataset redefined at reply input line 3.

Combining this with the functions to estimate the mean and standard deviation summary statistics, we can standardize our contrived dataset.

```

In [29]: $dataset = [[50, 30], [20, 90], [30, 50]];
printf "Means: %s\n", dump map {sprintf "%.4f", $_} @$means;
printf "Stdevs: %s\n", dump map {sprintf "%.4f", $_} @$stdevs;
# Standardize dataset
sml->standardize_dataset($dataset, $means, $stdevs);
printf "Normalized: %s\n", dump map { dump map {sprintf "%.4f", $_} @$_} @$dataset;

# Example Output From Standardizing the Contrived Dataset.
# Means: (33.33, 56.67)
# Stdevs: (15.28, 30.55)
# Standardized: ("(1.09, -0.87)", "(-0.87, 1.09)", "(-0.22, -0.22)")

```

Means: (33.3333, 56.6667)

Stdevs: (15.2753, 30.5505)

Normalized: ("(1.0911, -0.8729)", "(-0.8729, 1.0911)", "(-0.2182, -0.2182)")

Out[29]: 1

Again, we can demonstrate the standardization of a machine learning dataset. The example below demonstrates how to load and standardize the Pima Indians diabetes dataset, assumed to be in the current working directory as in the previous normalization example.

```

In [30]: # Load pima-indians-diabetes dataset
$filename = '../data/pima-indians-diabetes.csv';
$dataset = sml->load_csv($filename);

$X = [map { [ @$_[0 .. $#{$dataset->[0]} - 1] ] } @$dataset];
$y = [map { @$_[-1] } @$dataset];

# Calculate min and max for each column
$minmax = sml->dataset_minmax($X);
sml->normalize_dataset($X, $minmax);
# Estimate mean and standard deviation
$means = sml->column_means($X);
$stdevs = sml->column_stdevs($X, $means);
# standardize dataset
sml->standardize_dataset($X, $means, $stdevs);
printf "Standardized:%s\n", dump map {sprintf "%.4f", $_} @{$X->[0]};

# Example Output From Standardizing the Diabetes Dataset.
# Loaded data file ../data/pima-indians-diabetes.csv with 768 rows and 9 columns.
# Dataset[0]: ["6.0", "148.0", "72.0", "35.0", "0.0", 33.6, 0.6, "50.0", "1.0"]
# Dataset[0]: (0.64, 0.85, 0.15, 0.91, -0.69, "0.20", 0.38, 1.43, 1.37)

```

```
Standardized:(0.6395, 0.8478, 0.1495, 0.9067, -0.6924, 0.2039, 0.4682, 1.4251)
```

```
Out[30]: 1
```

2.2.3 When to Normalize and Standardize

Standardization is a scaling technique that assumes your data conforms to a normal distribution. If a given data attribute is normal or close to normal, this is probably the scaling method to use. It is good practice to record the summary statistics used in the standardization process so that you can apply them when standardizing data in the future that you may want to use with your model. Normalization is a scaling technique that does not assume any specific distribution.

If your data is not normally distributed, consider normalizing it prior to applying your machine learning algorithm. It is good practice to record the minimum and maximum values for each column used in the normalization process, again, in case you need to normalize new data in the future to be used with your model.

2.3 Extensions

There are many other data transforms you could apply. The idea of data transforms is to best expose the structure of your problem in your data to the learning algorithm. It may not be clear what transforms are required upfront. A combination of trial and error and exploratory data analysis (plots and stats) can help tease out what may work. Below are some additional transforms you may want to consider researching and implementing:

- Normalization that permits a configurable range, such as -1 to 1 and more.
- Standardization that permits a configurable spread, such as 1, 2 or more standard deviations from the mean.
- Exponential transforms such as logarithm, square root and exponents.
- Power transforms such as Box-Cox for fixing the skew in normally distributed data.

2.4 Review

In this tutorial, you discovered how to rescale your data for machine learning from scratch. Specifically, you learned:

- How to normalize data from scratch.
- How to standardize data from scratch.
- When to use normalization or standardization on your data.